Frontend Specification Document (FSD)

CyberLabX

Version: 1.0

Platform: Web (Desktop First)

Framework: React + TypeScript + TailwindCSS

---

1. Design Philosophy

CyberLabX should **not** look like a typical college ERP portal.

It should feel like a modern cybersecurity platform while remaining approachable for students who are new to security.

Design principles:

- Professional
- Minimal
- Dark-first
- High contrast
- Fast
- No unnecessary animations
- Accessibility compliant
- Terminal-inspired, not "hacker movie" inspired

Avoid the common mistake of using neon green text on black everywhere. Use those accents sparingly.

---

2. Brand Identity

Brand Personality

- Modern
- Intelligent
- Technical
- Trustworthy
- Clean

Keywords

```
None
Security
Precision
Learning
Innovation
Professional
```

---

3. Color System

Primary

Cyber Blue

```
None
#2563EB
```

Used for

- Primary buttons
- Links
- Selected items

Secondary

Emerald

```
None
#10B981
```

Used for

- Success
- Completed experiments
- Active labs

Accent

Cyber Cyan

```
None
#06B6D4
```

Used sparingly for:

- Progress indicators
- Charts
- Interactive highlights

Danger

```
None
#EF4444
```

Errors

Warning

```
None
#F59E0B
```

Warnings

Background

Primary

None
#0F172A

Secondary

None
#1E293B

Cards

None
#1F2937

Text

Primary

None
#F8FAFC

Secondary

None
#CBD5E1

Muted

None
#64748B

Borders

None
#334155

Terminal

Background

None
#0B0F19

Prompt

None
#22C55E

Command

None
#FFFFFF

Output

None
#94A3B8

Error

```
None
#EF4444
```

---

4. Typography

Font

Primary UI

Inter

Reasons

- Excellent readability
- Widely supported
- Modern SaaS feel

Terminal

JetBrains Mono

Reasons

- Excellent monospace font
- Easy to distinguish similar characters
- Popular among developers

Sizes

Element	Size
Hero	48px
H1	36px
H2	30px

H3	24px
H4	20px
Body	16px
Small	14px
Caption	12px

Line Height

```
None
1.5
```

---

5. Grid System

Desktop

```
None
12 Columns
```

Container

```
None
1440px
```

Content

```
None
1280px
```

---

6. Spacing System

Use an 8-point grid.

```
None
4
8
16
24
32
40
48
64
80
96
```

Never use arbitrary spacing.

---

7. Border Radius

Buttons

```
None
10px
```

Cards

```
None
16px
```

Inputs

```
None
10px
```

Modals

```
None
20px
```

Terminal

```
None
14px
```

---

8. Shadows

Cards

```
None
0 8px 20px rgba(0,0,0,.25)
```

Modals

```
None
0 20px 60px rgba(0,0,0,.45)
```

---

9. Components

Primary Button

Height

```
None
48px
```

Background

```
None
#2563EB
```

Text

White

Hover

```
None
#1D4ED8
```

Disabled

```
None
#475569
```

Loading

Spinner + Disabled

Secondary Button

Border

```
None
1px solid #334155
```

Transparent

Hover

```
None
#1F2937
```

Danger Button

Background

None
#EF4444

Inputs

Height

None
48px

Border

None
#334155

Focus

None
2px #2563EB

Placeholder

None
#64748B

Cards

Padding

None
24px

Radius

None
16px

Background

None
#1F2937

Border

None
1px #334155

Modal

Width

None
640px

Background

None
#111827

Overlay

None
rgba(0,0,0,.7)

Escape closes

Click outside closes

Sidebar

Collapsed

```
None
80px
```

Expanded

```
None
260px
```

---

10. Dashboard Layout

```
None
-----------------------------------------------------

Sidebar

|

|

Top Navbar

|

|

Content Grid
```

```
|

|

Footer

-------------------------------------------------
```

---

11. Terminal Design

The terminal is the centerpiece of the application.

```
None
+---------------------------------------------+

• • •

root@cyberlab:~$

_

+---------------------------------------------+
```

Features

- Copy
- Paste
- Clear
- Download session
- Fullscreen
- Resize

---

12. Experiment Page

```
None
Theory

Objectives

Requirements

↓

Launch Lab

↓

Terminal

↓

Tasks

↓

Progress

↓

Quiz

↓

Report
```

---

13. Dashboard Cards

Each experiment card displays:

```
None
Icon

Title

Difficulty

Estimated Time

Completion Status

Start Button
```

---

14. Progress Bar

Colors

Not Started

Gray

Running

Blue

Completed

Green

Failed

Red

---

15. Icons

Library

Lucide React

Reasons

- Lightweight
- Consistent style
- Tree-shakeable

---

16. Animations

Library

Framer Motion

Duration

```
None
200–300ms
```

Use only for:

- Page transitions
- Drawer
- Modal
- Toast

Avoid excessive animations.

---

17. Charts

Library

Recharts

Used for

- Student progress
- Completion rates
- Scores
- Faculty analytics

---

18. Responsive Breakpoints

```
None
Mobile

0–767

Tablet

768–1023

Laptop

1024–1439

Desktop

1440+
```

Version 1 is optimized for desktop because cybersecurity labs are difficult to use effectively on small screens. Mobile support can be limited to viewing dashboards and reports.

---

19. Accessibility

- WCAG AA contrast targets.
- Full keyboard navigation.
- Visible focus states.
- ARIA labels on interactive controls.
- Screen reader support where practical.

- Do not rely on color alone to convey status.

---

20. Third-Party Services & Integrations

For the MVP, keep external dependencies to a minimum.

Authentication

Internal JWT Service

Purpose:

- User authentication.
- Session management.

Frontend calls:

```
None
POST /api/auth/login
```

Request

```
JSON
{
  "email": "student@example.com",
  "password": "password123"
}
```

Response

```
JSON
{
  "user": {
    "id": "u123",
    "name": "Devesh",
    "role": "student"
```

```
  }
}
```

The JWT is stored in an HttpOnly cookie by the server, so the frontend does not receive or store it directly.

User Profile

None

```
GET /api/auth/me
```

Response

JSON

```
{
  "id":"u123",
  "name":"Devesh",
  "role":"student",
  "email":"student@example.com"
}
```

Experiments

None

```
GET /api/experiments
```

Response

JSON

```
[
  {
    "id":"exp1",
```

```
    "title":"Network Mapping with Nmap",
    "difficulty":"Easy",
    "estimatedTime":45,
    "status":"available"
  }
]
```

Experiment Details

None

```
GET /api/experiments/:id
```

Response

JSON

```
{
  "id":"exp1",
  "title":"Network Mapping with Nmap",
  "theory":"...",
  "objectives":[...],
  "tasks":[...]
}
```

Start Lab

None

```
POST /api/labs/start
```

Request

JSON

```
{
```

```
  "experimentId":"exp1"
}
```

Response

```JSON
{
  "sessionId":"lab123",
  "status":"starting"
}
```

Lab Status

```None
GET /api/labs/:sessionId
```

Response

```JSON
{
  "status":"running",
  "terminalReady":true
}
```

Terminal (WebSocket)

Endpoint

```None
ws://server/api/terminal
```

Client sends

```JSON
{
  "type":"execute",
  "sessionId":"lab123",
  "command":"nmap -sS 192.168.1.10"
}
```

Server streams

```JSON
{
  "type":"output",
  "data":"Starting Nmap..."
}
```

Completion

```JSON
{
  "type":"complete",
  "exitCode":0
}
```

Submit Task

```None
POST /api/tasks/:id/submit
```

Request

```JSON
{
  "command":"nmap -sS 192.168.1.10"
}
```

Response

```
JSON
{
  "correct":true,
  "score":10,
  "feedback":"SYN scan executed successfully."
}
```

Quiz

```
None
POST /api/quizzes/:id/submit
```

Response

```
JSON
{
  "score":8,
  "total":10,
  "passed":true
}
```

Reports

```
None
GET /api/reports/:id
```

Response

```
JSON
{
  "id":"r1",
  "score":92,
```

```
  "generatedAt":"2026-07-28T12:30:00Z",
  "downloadUrl":"/api/reports/r1/download"
}
```

Faculty Dashboard

None

```
GET /api/faculty/dashboard
```

Response

JSON

```
{
  "activeStudents":42,
  "completedLabs":135,
  "averageScore":86
}
```

---

21. State Management

- **TanStack Query:** Server state (API data, caching, retries).
- **Zustand:** Lightweight client state (theme, sidebar state, active session metadata).
- Avoid Redux for the MVP; it adds complexity without clear benefit.

---

22. Design Tokens

Centralize all colors, spacing, typography, border radii, and shadows in a shared design token file (or Tailwind theme). Components should never hardcode values like `#2563EB` directly. This ensures future rebranding or theming requires changing a single source of truth rather than editing dozens of components.

---

23. Frontend Performance Guidelines

- Lazy-load experiment pages.
- Code-split the terminal and analytics modules.
- Optimize icon imports (tree-shaking).
- Virtualize long tables (student lists, logs).
- Debounce search inputs.
- Cache experiment metadata with TanStack Query.
- Show skeleton loaders instead of blocking spinners where possible.

Architectural Recommendation

There is one change I would make before writing any UI.

Do **not** design the interface around "pages." Design it around **application states**.

For example, the Lab page isn't a single screen—it has distinct states:

- Lab not started.
- Lab starting.
- Lab running.
- Lab disconnected.
- Lab reconnecting.
- Lab completed.
- Lab expired.
- Lab failed.

Likewise, the Experiment page can be in draft, available, locked, assigned, or completed states.

Designing these states first will produce a more resilient interface than simply creating static page mockups. It also gives frontend developers a clear contract for how the UI should behave as the backend responds to different conditions. This state-driven approach is common in mature SaaS products and will make CyberLabX significantly easier to maintain as new experiments and features are added.